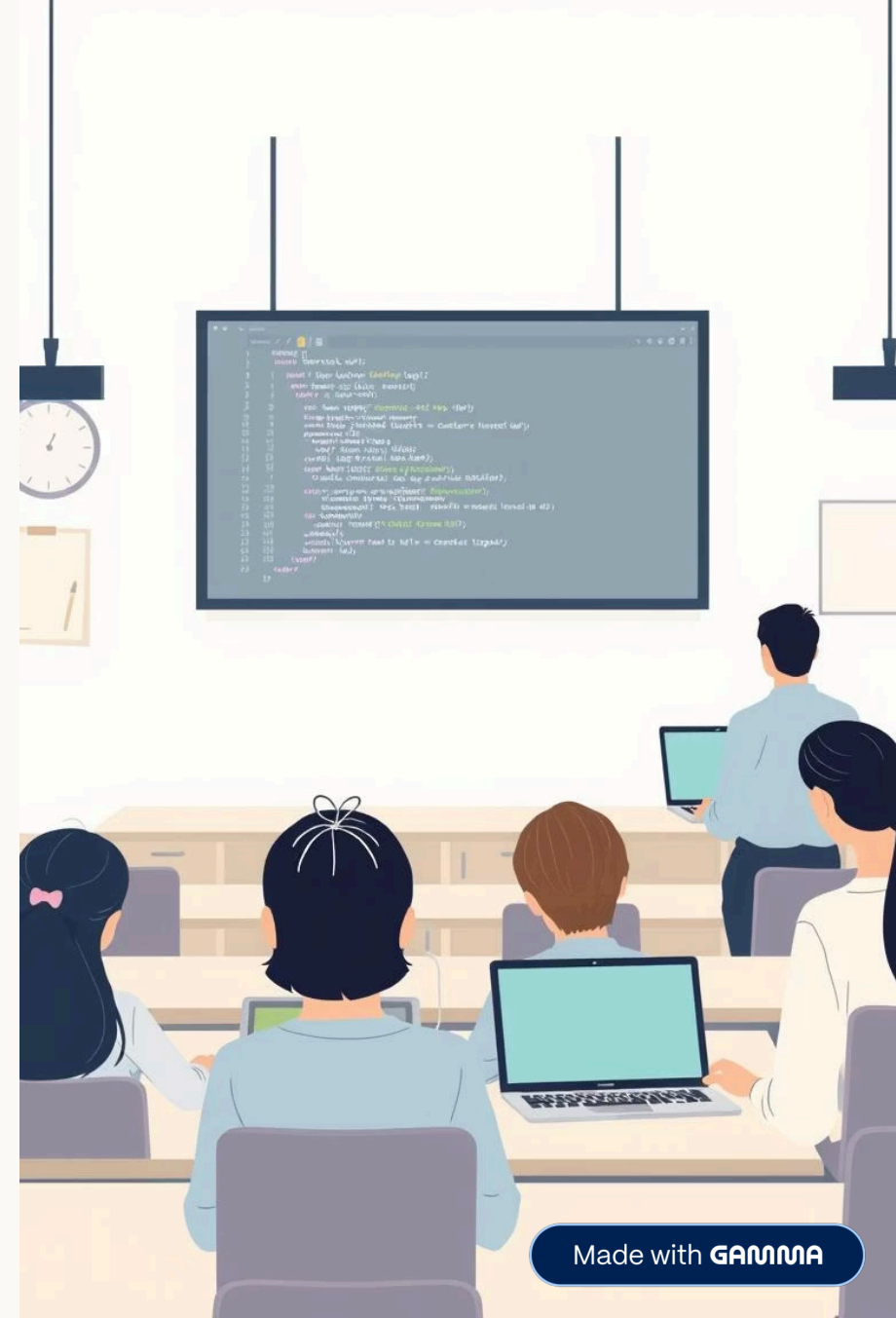


# Module 1 – Introduction à Java

Présenté par Raouf Ben Abdelwahed



# Objectifs du Module

Ce module est conçu pour vous offrir une base solide en programmation Java. À la fin de cette session, vous serez capable de :



## Comprendre l'écosystème Java

Différencier JVM, JDK et JRE.



## Écrire vos premiers programmes

Créer et exécuter un programme Java simple.



## Maîtriser les types de données

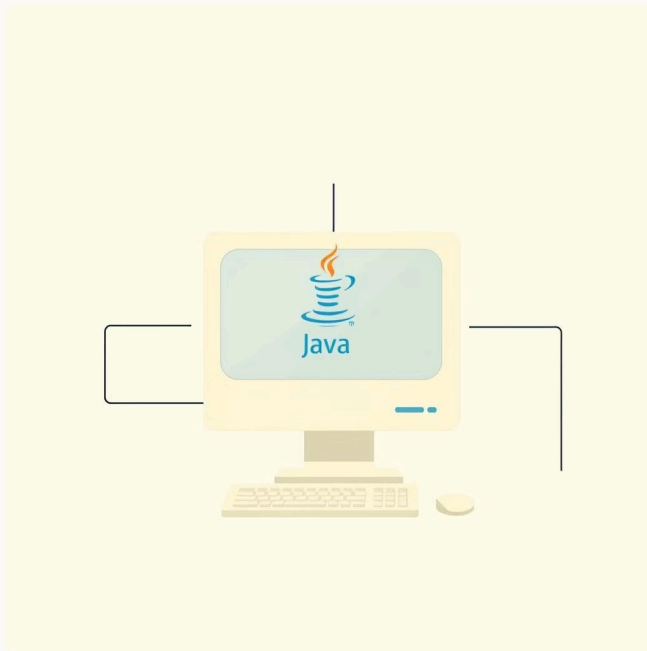
Déclarer et utiliser les types de données fondamentaux en Java.

# L'Écosystème Java

Java repose sur un ensemble de composants essentiels qui permettent le développement et l'exécution de vos applications.

## JRE (Java Runtime Environment)

C'est l'environnement nécessaire pour **exécuter** des applications Java. Il inclut la JVM et les bibliothèques principales.



## JVM (Java Virtual Machine)

La machine virtuelle qui permet à Java d'être multiplateforme. Elle **interprète** le bytecode Java en code machine.



## JDK (Java Development Kit)

L'ensemble d'outils complet pour **développer** des applications Java. Il contient le JRE, le compilateur (javac) et d'autres utilitaires.



# IDE et Installations Nécessaires

Pour commencer à coder en Java, il est essentiel de configurer votre environnement de développement. Voici les outils principaux dont vous aurez besoin.

## JDK (Java Development Kit)

Le JDK est le kit de développement Java. Il inclut le compilateur (javac) et l'environnement d'exécution (JRE), essentiels pour écrire et exécuter vos applications.



## IntelliJ IDEA

IntelliJ IDEA est un environnement de développement intégré (IDE) très apprécié pour sa productivité et ses fonctionnalités intelligentes. Il simplifie grandement le codage en Java.



## Ou Eclipse

Eclipse est un autre IDE open-source et très populaire pour le développement Java. Il est réputé pour sa flexibilité et son vaste écosystème de plugins.



# Votre Premier Programme Java : "Hello, World!"

C'est une tradition! Chaque développeur commence par ce programme simple. Il vous montre la structure de base d'un programme Java.

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

- **public class HelloWorld:** Définit une nouvelle classe nommée HelloWorld.
- **public static void main(String[] args):** Le point d'entrée de votre programme.
- **System.out.println("Hello, World!");** Affiche le message à l'écran.

# Les Types de Données primitifs

En Java, chaque variable doit avoir un type. Voici les types primitifs les plus couramment utilisés.

| Type    | Bytes | Description                                   | Exemple                  |
|---------|-------|-----------------------------------------------|--------------------------|
| byte    | 1     | [-128, 127]                                   | byte i = 0               |
| short   | 2     | [-32K, 32K]                                   | short m = 14             |
| int     | 4     | [-2B, 2B]                                     | int age = 30;            |
| long    | 8     |                                               |                          |
| float   | 4     |                                               |                          |
| double  | 8     | Nombres à virgule flottante (avec décimales). | double prix = 19.99;     |
| char    | 2     | Un seul caractère.                            | char initiale = 'J';     |
| boolean |       | Valeur logique : true ou false.               | boolean estActif = true; |
|         |       |                                               |                          |

Le choix du type approprié est crucial pour l'efficacité et la précision de votre code.

# Déclaration et Initialisation de Variables

Déclarer une variable, c'est lui donner un nom et un type. L'initialiser, c'est lui assigner une première valeur.

## Syntaxe de Déclaration :

```
type nomVariable;
```

## Syntaxe d'Initialisation :

```
nomVariable = valeur;
```

## Déclaration et Initialisation en une seule ligne :

```
type nomVariable = valeur;
```

```
public class Variables {  
    public static void main(String[] args) {  
        // Déclaration  
        int quantite;  
  
        // Initialisation  
        quantite = 100;  
  
        // Déclaration et initialisation  
        double solde = 500.75;  
  
        System.out.println("Quantité : " + quantite);  
        System.out.println("Solde : " + solde);  
    }  
}
```

# Les Types de Données Reference

En Java, il existe aussi les types **reference**:

- commence par une majuscule: **String, Date..**
- **déclaré avec new**
- possède des méthodes propres: *exemple* → `"hello".toUpperCase();`

nécessite généralement d'importer une bibliothèque: `import java.util.Date;`

Le choix du type approprié est crucial pour l'efficacité et la précision de votre code.

Exemple:

```
String phrase = "ma phrase est bien écrite";
```

```
Date now = new Date();
```

```
System.out.println(now);
```

```
System.out.println(phrase + "!!!");
```

```
System.out.println(now);
```



# Les Opérateurs en Java

Les opérateurs sont des symboles qui réalisent des opérations sur des valeurs ou des variables.

## Opérateurs Arithmétiques

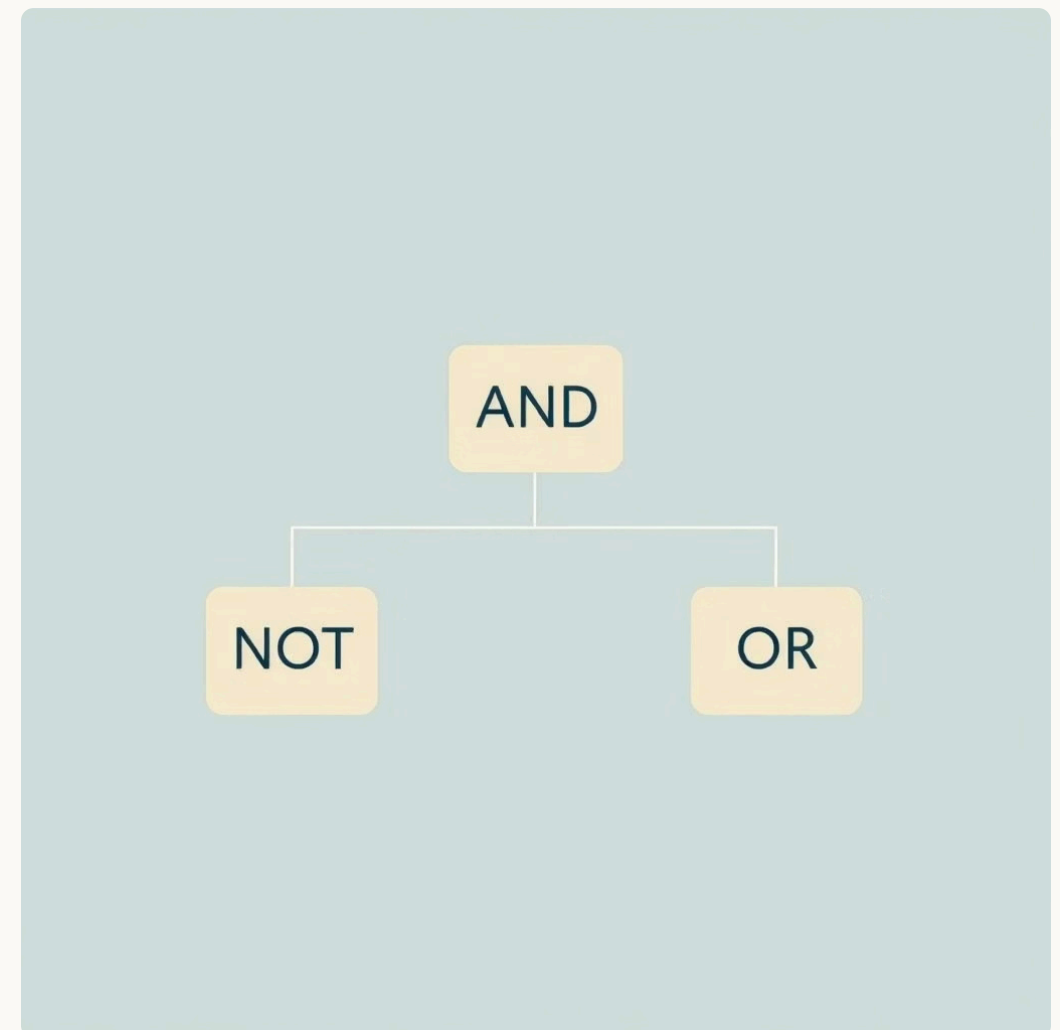
- **+** (Addition)
- **-** (Soustraction)
- **\*** (Multiplication)
- **/** (Division)
- **%** (Modulo - reste de la division)

## Opérateurs Relationnels

- **==** (Égal à)
- **!=** (Différent de)
- **<** (Inférieur à)
- **>** (Supérieur à)
- **<=** (Inférieur ou égal à)
- **>=** (Supérieur ou égal à)

## Opérateurs Logiques

- **&&** (ET logique)
- **||** (OU logique)
- **!** (NON logique)



# Exercice Pratique

Mettez en application ce que vous avez appris en créant un programme Java qui effectue des calculs simples.

## Déclarer deux entiers

Choisissez deux nombres entiers de votre choix et déclarez-les.

## Calculer Somme et Produit

Utilisez les opérateurs arithmétiques pour trouver leur somme et leur produit.

## Calculer la Moyenne

Calculez la moyenne de ces deux nombres. Pensez au type de données pour le résultat!

## Afficher les Résultats

Utilisez `System.out.println()` pour afficher la somme, le produit et la moyenne.

# Mini Quiz : Testez vos Connaissances !

Vérifions votre compréhension des concepts clés abordés jusqu'à présent.

## Question 1

Quel composant Java est nécessaire pour **exécuter** un programme Java sans le développer ?



## Question 2

Quel est le type de données approprié pour stocker un nombre décimal comme 3.14 ?



## Question 3

Quel opérateur logique est utilisé pour vérifier si **deux** conditions sont vraies simultanément ?

# Les Tableaux (Arrays) en Java

Les tableaux sont des structures de données fondamentales en Java, conçues pour stocker une collection de valeurs du même type sous un seul nom. Ils sont essentiels pour organiser des données de manière séquentielle.

## Type Référence

Contrairement aux types primitifs, les tableaux sont des **objets** en Java. Ils sont stockés dans le tas (heap) et vous accédez à leurs éléments via une référence.

## Taille Fixe

Une caractéristique importante des tableaux est que leur taille est **fixe** une fois qu'ils sont créés. Vous devez définir leur capacité dès la déclaration et elle ne peut pas être modifiée par la suite.

## Déclaration et Initialisation

Il existe deux façons principales de déclarer et d'initialiser un tableau en Java :

```
// Syntaxe 1: Déclaration avec taille spécifiée (les éléments sont initialisés par défaut)
type[] nomTableau = new type[taille];
```

```
// Syntaxe 2: Déclaration et initialisation avec des valeurs directes
type[] nomTableau = {valeur1, valeur2, valeur3, ...};
```

## Exemple Pratique

Comprenez comment les tableaux sont utilisés avec cet exemple simple :

```
public class TableauExemple {
    public static void main(String[] args) {
        // Déclaration et initialisation d'un tableau d'entiers
        int[] scores = {85, 92, 78, 95, 88};

        // Accès aux éléments du tableau (l'index commence à 0)
        System.out.println("Score du premier étudiant : " + scores[0]);
        System.out.println("Score du troisième étudiant : " + scores[2]);

        // Modifier un élément du tableau
        scores[4] = 90;
        System.out.println("Score du dernier étudiant après modification : " + scores[4]);

        // Obtenir la taille du tableau
        System.out.println("Nombre total de scores : " + scores.length);
        System.out.println("Exemple 2 : ");
        int[] nombres = new int[5];
        nombres[0] = 12;
        nombres[1] = -9;
        nombres[2] = 103;
        System.out.println(nombres);
        System.out.println(Arrays.toString(nombres));
        System.out.println(nombres.length);
        Arrays.sort(nombres);
        System.out.println(Arrays.toString(nombres));
    }
}
```

Cette structure vous permet de manipuler efficacement des collections de données homogènes.

# Les Structures Conditionnelles en Java

Les structures conditionnelles sont des outils fondamentaux en programmation Java qui vous permettent de prendre des décisions dans votre code. Elles contrôlent le flux d'exécution en fonction de la validité de certaines conditions, permettant ainsi à votre programme de réagir de manière différente selon les données d'entrée ou l'état de l'application.



if

Exécute un bloc de code **uniquement** si une condition spécifiée est vraie.



if-else

Exécute un bloc de code si la condition est vraie, et un **autre** bloc si elle est fausse.



if-else if-else

Permet de tester **plusieurs conditions** les unes après les autres jusqu'à ce qu'une soit vraie, ou d'exécuter un bloc par défaut.



switch

Évalue une expression et exécute le bloc de code correspondant à la **valeur de cette expression**.

## Exemple Pratique : Système de Notation

Voici comment utiliser une structure `if-else if-else` pour déterminer la note d'un étudiant en fonction de son score :

```
public class SystemeNotation {
    public static void main(String[] args) {
        int score = 85; // Supposons un score de 85

        if (score >= 90) {
            System.out.println("Note : A");
        } else if (score >= 80) {
            System.out.println("Note : B");
        } else if (score >= 70) {
            System.out.println("Note : C");
        } else if (score >= 60) {
            System.out.println("Note : D");
        } else {
            System.out.println("Note : F");
        }
    }
}
```

Les structures conditionnelles sont essentielles pour créer des programmes Java intelligents et réactifs, capables de s'adapter à diverses situations.

# Les Boucles en Java

Les boucles sont des constructions essentielles en Java, vous permettant d'exécuter un bloc de code de manière répétée. Elles sont indispensables pour automatiser des tâches, parcourir des collections de données ou attendre qu'une condition soit remplie.

|                                                            |                                                                                     |                                                                            |
|------------------------------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>Répétition</b><br>Exécuter le même code plusieurs fois. | <b>Itération</b><br>Parcourir les éléments d'une collection (tableau, liste, etc.). | <b>Condition</b><br>Continuer l'exécution tant qu'une condition est vraie. |
|------------------------------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------|

## La boucle for

La boucle `for` est utilisée lorsque vous savez à l'avance combien de fois vous souhaitez répéter un bloc de code. Elle est idéale pour les itérations basées sur un compteur.

```
// Afficher les nombres de 0 à 4
for (int i = 0; i < 5; i++) {
    System.out.println("Compteur : " + i);
}
```

## La boucle while

La boucle `while` exécute un bloc de code tant qu'une condition spécifiée reste vraie. Elle est utilisée lorsque le nombre d'itérations n'est pas connu à l'avance.

```
// Compter tant que 'compteur' est inférieur à 3
int compteur = 0;
while (compteur < 3) {
    System.out.println("Le compteur est à : " + compteur);
    compteur++;
}
```

## La boucle do-while

La boucle `do-while` est similaire à `while`, mais elle garantit que le bloc de code est exécuté au moins une fois, avant de vérifier la condition.

```
// Exécuter une fois, puis vérifier la condition
int j = 0;
do {
    System.out.println("Le do-while est à : " + j);
    j++;
} while (j < 1);
```

## La boucle for-each (Boucle For Améliorée)

Introduite avec Java 5, la boucle `for-each` simplifie l'itération sur les collections et les tableaux. Elle est plus concise et moins sujette aux erreurs d'indice.

```
// Parcourir un tableau d'entiers
int[] nombres = {10, 20, 30, 40, 50};
for (int nombre : nombres) {
    System.out.println("Élément : " + nombre);
}
```

# Les Matrices (Tableaux 2D) en Java

En Java, une **matrice**, ou tableau bidimensionnel (2D), est un tableau dont chaque élément est lui-même un tableau. Cette structure est idéale pour représenter des données tabulaires, des grilles ou toute information organisée en lignes et colonnes.

|    |     |     |    |     |     |    |     |    |     |    |     |     |    |    |    |
|----|-----|-----|----|-----|-----|----|-----|----|-----|----|-----|-----|----|----|----|
|    | 109 | 109 |    | 217 | 190 |    | 215 |    | 111 |    | 217 | 227 | 49 |    | 38 |
| 1  | 1   | 15  | 2  | 4   | 15  | 4  | 5   | 10 | 4   | 5  | 15  | 15  | 10 | 1  | 15 |
| 2  | 10  | 13  | 15 | 10  | 15  | 4  | 15  | 15 | 15  | 15 | 13  | 15  | 15 | 5  | 15 |
| 3  | 20  | 15  | 15 | 4   | 16  | 5  | 15  | 18 | 14  | 25 | 15  | 18  | 15 | 5  | 15 |
| 4  | 15  | 15  | 15 | 5   | 16  | 1  | 15  | 15 | 15  | 15 | 15  | 15  | 15 | 6  | 15 |
| 5  | 16  | 15  | 16 | 6   | 15  | 5  | 15  | 15 | 16  | 15 | 15  | 15  | 15 | 8  | 15 |
| 6  | 16  | 14  | 15 | 6   | 15  | 7  | 25  | 15 | 15  | 5  | 15  | 16  | 15 | 5  | 26 |
| 7  | 15  | 15  | 15 | 16  | 16  | 7  | 16  | 15 | 19  | 14 | 15  | 16  | 15 | 5  | 16 |
| 8  | 15  | 15  | 15 | 6   | 15  | 9  | 16  | 19 | 15  | 11 | 15  | 15  | 15 | 10 | 25 |
| 9  | 10  | 15  | 25 | 5   | 16  | 7  | 15  | 15 | 18  | 15 | 15  | 15  | 15 | 5  | 25 |
| 10 | 15  | 15  | 15 | 15  | 15  | 16 | 15  | 15 | 19  | 15 | 15  | 15  | 15 | 10 | 15 |
| 11 | 10  | 15  | 20 | 5   | 15  | 3  | 16  | 15 | 18  | 25 | 16  | 15  | 15 | 5  | 16 |
| 12 | 26  | 15  | 15 | 15  | 15  | 5  | 10  | 15 | 15  | 15 | 16  | 15  | 15 | 10 | 15 |
| 13 | 25  | 18  | 10 | 5   | 16  | 15 | 25  | 16 | 15  | 9  | 13  | 16  | 13 | 5  | 16 |
| 14 | 16  | 15  | 16 | 17  | 15  | 14 | 26  | 15 | 18  | 15 | 15  | 14  | 15 | 15 | 16 |
| 15 | 15  | 19  | 22 | 16  | 15  | 15 | 25  | 21 | 15  | 16 | 15  | 16  | 15 | 25 | 30 |

Matrice de 15 lignes  
par 15 colonnes

## Déclaration et Initialisation

Vous pouvez déclarer et initialiser une matrice en spécifiant le nombre de lignes et de colonnes, ou en fournissant directement les valeurs :

```
// Déclaration d'une matrice de 2 lignes et 3 colonnes
int[][] nombres = new int[2][3];

// Initialisation des éléments spécifiques
nombres[0][0] = 12;
nombres[0][1] = -9;
nombres[1][0] = 103;

// Affichage du tableau 2D
System.out.println("Arrays.toString(nombres) : " + Arrays.toString(nombres));
System.out.println("Arrays.deepToString(nombres) : " + Arrays.deepToString(nombres));
```

 **Important : Affichage des Matrices**

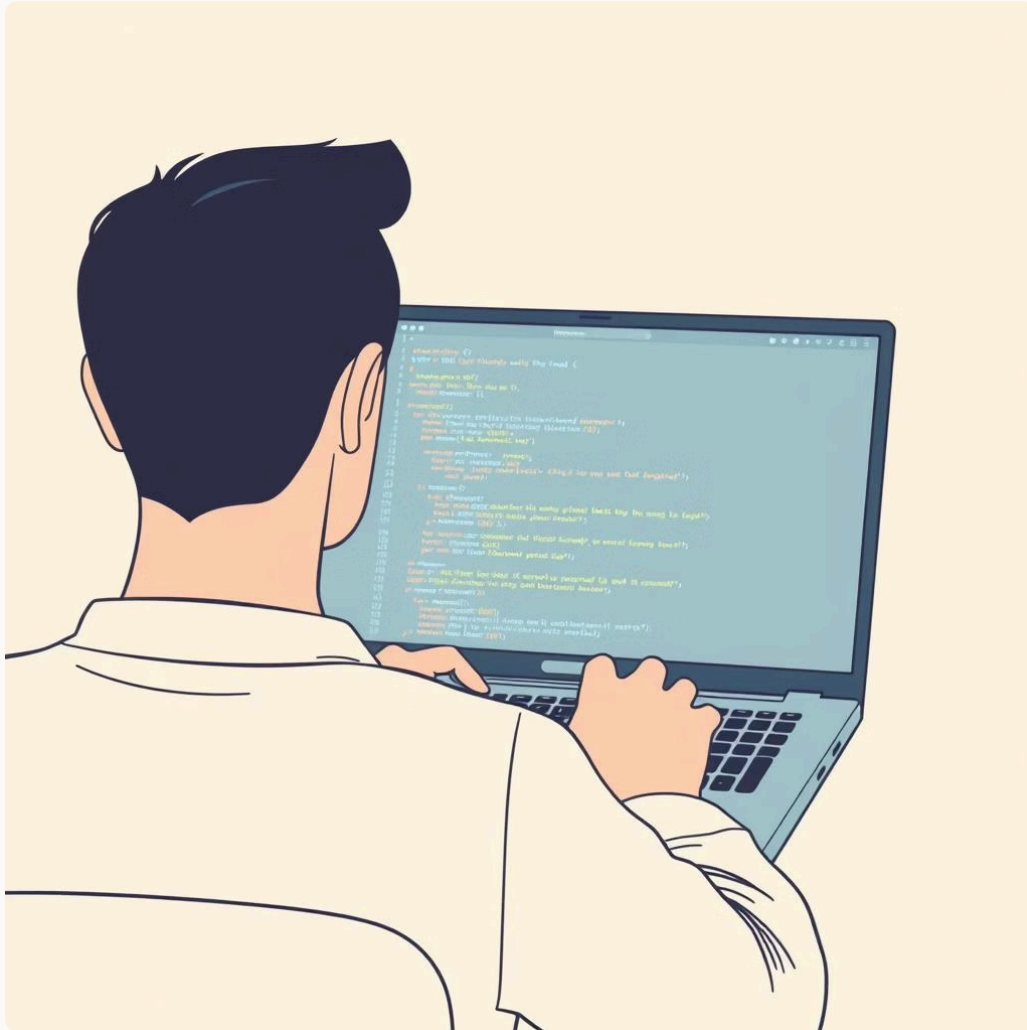
Contrairement aux tableaux unidimensionnels, pour afficher correctement le contenu d'une matrice (tableau 2D), il faut utiliser `Arrays.deepToString()`. La méthode `Arrays.toString()` n'affichera que les références mémoire des tableaux internes, et non leurs valeurs réelles.

Les matrices sont fondamentales pour manipuler des ensembles de données complexes de manière organisée et efficace en Java.



# Applications Pratiques

Mettez en pratique vos connaissances en résolvant les problèmes de programmation suivants :



- Écrire un programme qui affiche un tableau de 20 entiers contenant les valeurs de 0 à 19.
- Écrire un programme qui construit une matrice dont les éléments sont la somme des compteurs  $i+j$  (exemple: `matrix[1][3] = 1+3 = 4`).
- Compléter le programme précédent pour afficher combien de 5 il y a dans la matrice.
- Soient deux variables `double n1, n2`, écrire un programme qui affiche le maximum entre ces deux nombres réels.



# Saisie des Données Utilisateur avec Scanner

Le programme Java interagit souvent avec l'utilisateur. La classe `Scanner` est l'outil principal pour lire les données saisies au clavier, qu'il s'agisse de nombres, de chaînes de caractères ou d'autres types d'informations.

## Exemple d'Interaction Utilisateur

```
import java.util.Scanner; // N'oubliez pas d'importer la classe Scanner

public class InteractionUtilisateur {
    public static void main(String[] args) {
        String name;
        byte age;

        Scanner scanner = new Scanner(System.in); // Création d'un objet Scanner

        System.out.println("Comment vous appelez-vous ?");
        name = scanner.next(); // Lit le prénom (un seul mot)

        System.out.println("Quel est votre âge ?");
        age = scanner.nextByte(); // Lit l'âge (un octet)

        System.out.println("Bonjour " + name + ", vous avez " + age + " ans.");

        scanner.close(); // Fermeture du scanner pour libérer les ressources
    }
}
```

### ⚠ Comprendre `scanner.next()`

Lorsque vous exécutez ce programme :

- Si vous entrez un **prénom simple** (ex: "Alice"), le programme fonctionnera comme prévu.
- Si vous entrez un **nom complet** (ex: "Alice Wonderland"), `scanner.next()` ne lira que "Alice". Le reste ("Wonderland") sera laissé dans le buffer d'entrée et pourrait causer des comportements inattendus lors des lectures suivantes.

Pour lire une ligne entière, y compris les espaces, utilisez `scanner.nextLine()`.



La maîtrise de la classe `Scanner` est fondamentale pour créer des applications Java interactives et conviviales.

# Les Fonctions et Procédures en Java

En Java, les **fonctions** (appelées méthodes) et **procédures** (méthodes void) sont des blocs de code réutilisables qui effectuent des tâches spécifiques. Elles améliorent l'organisation, la lisibilité et la maintenabilité de votre code.



## Fonctions (Méthodes avec retour)

Effectuent une tâche et renvoient une valeur.



## Procédures (Méthodes void)

Exécutent une tâche sans renvoyer de valeur.

## Définition et Utilisation

Chaque méthode peut prendre des paramètres en entrée et, si c'est une fonction, retourner un résultat.

// Exemple de fonction (additionne deux nombres)

```
public int additionner(int a, int b) {  
    return a + b;  
}
```

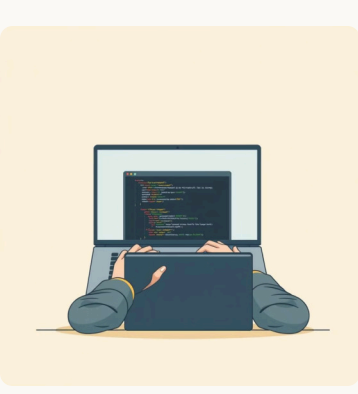
// Exemple de procédure (affiche un message)

```
public void saluer(String nom) {  
    System.out.println("Bonjour, " + nom + " !");  
}
```

// appeler dans main



Est ce que main est une procédure ou une méthode ? (**attention au piège !**)



# Mini Quiz Java – Module 1

## Testez vos connaissances sur les concepts fondamentaux !

### Question 1 – Variables et Types de données

Parmi les déclarations suivantes, laquelle est **correcte** ?

- A) `int nombre = 3.14;`
- B) `double prix = 19.99;`
- C) `String nom = 123;`
- D) `boolean actif = "true";`

### Question 2 – Structures conditionnelles

Que va afficher ce code ?

```
int age = 17;
if (age >= 18) {
    System.out.println("Majeur");
} else if (age >= 16) {
    System.out.println("Presque majeur");
} else {
    System.out.println("Mineur");
}
```

- A) Majeur
- B) Presque majeur
- C) Mineur
- D) Erreur de compilation

### Question 3 – Boucles

Combien de fois s'exécute cette boucle ?

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

- A) 4 fois
- B) 5 fois
- C) 6 fois
- D) Infiniment

### Question 4 – Tableaux

Comment déclarer un tableau d'entiers de 10 éléments en Java ?

- A) `int[] tableau = new int[10];`
- B) `int tableau[10];`
- C) `array<int> tableau = new array<int>{10};`
- D) `int tableau = new int[10];`

### Question 5 – Méthodes

Quelle est la signature correcte d'une méthode qui retourne la somme de deux entiers ?

- A) `public void somme(int a, int b)`
- B) `public int somme(int a, int b)`
- C) `somme(int a, int b) returns int`
- D) `public static somme(a, b): int`

### Question 6 – Scanner

Pour lire un entier saisi par l'utilisateur avec Scanner, on utilise :

- A) `scanner.readInt()`
- B) `scanner.nextIntInteger()`
- C) `scanner.nextInt()`
- D) `scanner.getInt()`

### Question 7 – Tableaux 2D (Matrices)

Comment accéder à l'élément de la 2ème ligne, 3ème colonne d'une matrice `int[][] matrice` ?

- A) `matrice[2][3]`
- B) `matrice[1][2]`
- C) `matrice(2,3)`
- D) `matrice[3][2]`

### Question 8 – Boucle while

Que va afficher ce code ?

```
int i = 1;
while (i <= 3) {
    System.out.print(i + " ");
    i++;
}
```

- A) 1 2 3
- B) 1 2 3 4
- C) 0 1 2 3
- D) 2 3 4

### Question 9 – Opérateurs

Quelle est la valeur de x après cette instruction ?

```
int x = 10;
x += 5;
```

- A) 10
- B) 5
- C) 15
- D) 50

### Question 10 – Méthodes et portée des variables

Dans ce code, que va afficher la méthode main ?

```
public class Test {
    static int nombre = 5;

    public static void modifier() {
        nombre = 10;
    }

    public static void main(String[] args) {
        modifier();
        System.out.println(nombre);
    }
}
```

- A) 5
- B) 10
- C) 0
- D) Erreur de compilation

## Réponses et Explications

▼ Cliquez ici pour voir les réponses

#### Réponses :

- B** - `double prix = 19.99;` est correct (double peut contenir des décimales)
- B** - "Presque majeur" (age=17, condition `age >= 16` est vraie)
- B** - 5 fois (i va de 0 à 4 inclus)
- A** - `int[] tableau = new int[10];` est la syntaxe correcte
- B** - `public int somme(int a, int b)` retourne un entier
- C** - `scanner.nextInt()` pour lire un entier
- B** - `matrice[1][2]` (indices commencent à 0)
- A** - "1 2 3 " (i va de 1 à 3)
- C** - 15 (`x = x + 5 = 10 + 5`)
- B** - 10 (la variable statique est modifiée par la méthode)

#### Barème :

- 9-10 bonnes réponses** : Excellent ! 🏆
- 7-8 bonnes réponses** : Très bien ! 👍
- 5-6 bonnes réponses** : Bien, mais révisez certains points
- 0-4 bonnes réponses** : Il faut revoir les concepts de base

## Concepts clés couverts dans le projet :

- ✅ **Variables et types de données** : `int`, `double`, `String`, `boolean`
- ✅ **Structures conditionnelles** : `if/else`, `switch`
- ✅ **Boucles** : `for`, `while`, `do-while`
- ✅ **Tableaux 1D et 2D** : Déclaration, initialisation, manipulation
- ✅ **Méthodes** : Définition, paramètres, valeurs de retour
- ✅ **Scanner** : Interaction avec l'utilisateur
- ✅ **Validation des données** : Gestion des erreurs de saisie
- ✅ **Algorithmes de tri** : Tri par sélection
- ✅ **Constantes** : `final` `static`